Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»
Кафедра электронных вычислительных машин

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 6
Concurrency
по дисциплине «Программирование на языках высокого уровня»

Выполнил ст. гр. 450502 Е.Ю. Новицкий

Проверил асс. каф. ЭВМ И.Г. Скиба

Минск 2026

1 ПОСТАНОВКА ЗАДАЧИ

1. Реализовать асинхронную бизнес-операцию через @Async / CompletableFuture, которая:

– возвращает ID задачи

– позволяет проверить статус выполнения

2. Реализовать потокобезопасный счётчик (или аналогичный механизм) с использованием synchronized или Atomic.

3. Продемонстрировать возможный race condition (ExecutorService 50+ потоков) и его решение.

4. Провести нагрузочное тестирование JMeter и показать результаты.

2 СТРУКТУРА ПРОЕКТА

```
.
├── README.md
├── config
│   └── checkstyle
│       └── checkstyle.xml
├── docs
│   ├── ER-diagram.png
│   ├── api.md
│   ├── jmeter
│   │   ├── all-endpoints.jmx
│   │   └── lab6-race-condition.jmx
│   ├── lab6-demo.md
│   └── lab6-theory.md
├── logs
│   ├── application.log
│   └── error.log
├── mvnw
├── mvnw.cmd
├── pom.xml
└── src
    ├── main
    │   ├── java
    │   │   └── com
    │   │       └── finance
    │   │           └── tracker
    │   │               ├── aop
    │   │               │   └── ServiceLoggingAspect.java
    │   │               ├── cache
    │   │               │   ├── TransactionSearchCacheKey.java
    │   │               │   ├── TransactionSearchIndex.java
    │   │               │   └── TransactionSearchIndexInvalidator.java
    │   │               ├── config
    │   │               │   ├── AsyncConfig.java
    │   │               │   └── OpenApiConfig.java
    │   │               ├── controller
    │   │               │   ├── AccountController.java
    │   │               │   ├── AsyncTransactionController.java
    │   │               │   ├── BudgetController.java
    │   │               │   ├── CategoryController.java
    │   │               │   ├── RaceConditionController.java
    │   │               │   ├── TransactionController.java
    │   │               │   ├── UserController.java
    │   │               │   └── api
    │   │               │       ├── AccountControllerApi.java
    │   │               │       ├── AsyncTransactionControllerApi.java
    │   │               │       ├── BudgetControllerApi.java
    │   │               │       ├── CategoryControllerApi.java
    │   │               │       ├── RaceConditionControllerApi.java
    │   │               │       ├── TransactionControllerApi.java
    │   │               │       └── UserControllerApi.java
    │   │               ├── domain
    │   │               │   ├── Account.java
    │   │               │   ├── AccountType.java
    │   │               │   ├── AsyncTaskStatus.java
    │   │               │   ├── AsyncTransactionTask.java
    │   │               │   ├── Budget.java
    │   │               │   ├── Category.java
    │   │               │   ├── Transaction.java
    │   │               │   ├── TransactionType.java
    │   │               │   └── User.java
    │   │               ├── dto
    │   │               │   ├── request
    │   │               │   └── response
    │   │               ├── exception
    │   │               ├── logging
    │   │               ├── mapper
    │   │               ├── repository
    │   │               ├── service
```

```
│   │   │   │   ├── AccountService.java
│   │   │   │   ├── AsyncTransactionService.java
│   │   │   │   ├── BudgetService.java
│   │   │   │   ├── CategoryService.java
│   │   │   │   ├── CounterService.java
│   │   │   │   ├── TransactionService.java
│   │   │   │   ├── UserService.java
│   │   │   │   └── impl
│   │   │   │       ├── AccountServiceImpl.java
│   │   │   │       ├── AsyncTaskStorage.java
│   │   │   │       ├── AsyncTransactionExecutorService.java
│   │   │   │       ├── AsyncTransactionServiceImpl.java
│   │   │   │       ├── BudgetServiceImpl.java
│   │   │   │       ├── CategoryServiceImpl.java
│   │   │   │       ├── CounterServiceImpl.java
│   │   │   │       ├── RaceConditionDemoService.java
│   │   │   │       ├── TransactionServiceImpl.java
│   │   │   │       └── UserServiceImpl.java
│   │   │   └── FinanceTrackerApplication.java
│   │   └── resources
│   │       ├── application.properties
│   │       ├── db
│   │       │   ├── db.changelog-master.yaml
│   │       │   └── migrations
│   │       └── logback-spring.xml
└── test
    ├── java
    │   └── com
    │       └── finance
    │           └── tracker
    │               ├── controller
    │               ├── logging
    │               └── service
    │                   └── impl
    └── resources
        ├── application.properties
        ├── logback-test.xml
        └── mockito-extensions
            └── org.mockito.plugins.MockMaker
```

3 ЛИСТИНГ КОДА

Файл AsyncTransactionController.java

```
package com.finance.tracker.controller;

import com.finance.tracker.domain.AsyncTask;
import com.finance.tracker.dto.request.TransactionRequest;
import com.finance.tracker.service.impl.TransactionServiceImpl;
import jakarta.validation.Valid;
import java.util.List;
import java.util.Map;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/transactions/async")
@RequiredArgsConstructor
public class AsyncTransactionController {

    private final TransactionServiceImpl transactionService;

    @PostMapping
    public ResponseEntity<Map<String, String>> createTransactionsAsync(
            @Valid @RequestBody List<@Valid TransactionRequest> requests,
            @RequestParam(defaultValue = "true") boolean transactional) {
        String taskId = transactionService.createTransactionsAsync(requests, transactional);
        return ResponseEntity.accepted().body(Map.of("taskId", taskId));
    }

    @GetMapping("/status/{taskId}")
    public ResponseEntity<AsyncTask> getTaskStatus(@PathVariable String taskId) {
        AsyncTask task = transactionService.getTransactionTaskStatus(taskId);
        if (task == null) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(task);
    }

    @GetMapping("/tasks")
    public ResponseEntity<Map<String, AsyncTask>> getAllAsyncTasks() {
        return ResponseEntity.ok(transactionService.getAllAsyncTasks());
    }
}
```

Файл RaceCondition.java

```
package com.finance.tracker.controller;

import com.finance.tracker.service.impl.RaceConditionDemoService;
import java.util.HashMap;
import java.util.Map;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/demo/race-condition")
@RequiredArgsConstructor
public class RaceConditionController {

    private final RaceConditionDemoService demoService;

    @GetMapping("/run")
    public ResponseEntity<Map<String, String>> runRaceConditionDemo() throws InterruptedException
{
```

```
        demoService.runAllDemos();

        Map<String, String> response = new HashMap<>();
        response.put("message", "Демонстрация race condition запущена");
        response.put("status", "Проверьте логи для результатов");

        return ResponseEntity.ok(response);
    }
}
```

Файл AsyncConfig.java

```
package com.finance.tracker.config;

import java.util.concurrent.Executor;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.AsyncConfigurer;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
        executor.setQueueCapacity(25);
        executor.setThreadNamePrefix("AsyncTask-");
        executor.initialize();
        return executor;
    }
}
```

Файл CounterService.java

```
package com.finance.tracker.service;

public interface CounterService {

    void increment();

    int getValue();

    void reset();
}
```

Файл AsyncTask

```
package com.finance.tracker.domain;

import java.time.LocalDateTime;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class AsyncTask {

    private String taskId;
    private AsyncTaskStatus status;
    private LocalDateTime startTime;
    private LocalDateTime endTime;
    private Integer progress;
    private String result;
```

```
}
```

Файл AsyncTaskStatus

```
package com.finance.tracker.domain;

public enum AsyncTaskStatus {
    PENDING,
    IN_PROGRESS,
    COMPLETED,
    FAILED
}
```

4 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

GET {{baseUrl}} /transactions/bulk/async/ {{taskId}}

Docs Params Authorization Headers (6) Body Scripts Settings

Body Cookies Headers (5) Test Results 200 OK

{ } JSON Preview Visualize


```
1  {
2      "taskId": "4559b253-14da-412d-9e13-4d99e91b5c86",
3      "status": "COMPLETED",
4      "transactional": true,
5      "totalItems": 3,
6      "processedItems": 3,
7      "progressPercent": 100,
8      "createdTransactionIds": [
9          3313,
10         3314,
11         3315
12     ],
13     "summary": "Created 3 transactions",
14     "errorMessage": null,
15     "createdAt": "2026-04-06T15:56:39.340753",
16     "startedAt": "2026-04-06T15:56:39.342628",
17     "completedAt": "2026-04-06T15:56:42.472553"
18 }
```


Рисунок 4.1 – Демонстрация асинхронного bulk запроса

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB...	Sent KB/sec	Avg. Bytes
GET /budgets	50	28	17	67	11.47	0.00%	27.0/sec	1051.41	4.85	39900.4
GET /budgets/{id}	50	2	1	13	2.23	0.00%	28.0/sec	8.90	5.18	325.0
POST /categories	50	4	2	18	2.98	0.00%	27.9/sec	6.46	7.88	237.0
GET /categories	50	10	4	70	9.58	0.00%	26.9/sec	126.46	4.92	4809.6
GET /categories/{id}	50	2	1	10	1.82	0.00%	27.9/sec	6.47	5.29	237.0
PUT /categories/{id}	50	4	2	17	2.82	0.00%	27.9/sec	6.65	8.23	244.0
PUT /budgets/{id}	50	5	3	14	2.83	0.00%	28.0/sec	9.24	10.22	338.0
PATCH /budgets/{id}	50	5	3	24	4.59	0.00%	27.9/sec	9.21	9.51	338.0
POST /accounts/transf...	50	2	1	10	1.72	0.00%	28.0/sec	7.41	8.67	271.0
POST /transactions	50	2	1	8	1.17	0.00%	28.0/sec	11.26	9.87	412.0
GET /transactions	50	13	8	29	4.37	0.00%	27.7/sec	515.45	5.11	19065.0
GET /transactions?wit...	50	3	2	12	1.66	0.00%	28.0/sec	582.88	5.73	21344.2
GET /transactions by d...	50	13	7	32	5.24	0.00%	27.6/sec	426.51	6.87	15836.5
GET /transactions/{id}	50	2	1	11	1.91	0.00%	28.0/sec	11.26	5.30	412.0
GET /transactions/sear...	50	5	1	22	3.83	0.00%	27.9/sec	44.14	11.03	1617.3
POST /transactions/bu...	50	3	1	11	2.00	0.00%	27.9/sec	11.35	10.61	417.0
PUT /transactions/{id}	50	3	1	16	2.43	0.00%	27.8/sec	11.28	10.00	415.0
PATCH /transactions/{...	50	3	1	16	2.84	0.00%	27.8/sec	11.34	7.72	417.0
POST /transactions/bu...	50	1	0	5	0.96	0.00%	28.0/sec	8.63	10.84	316.0
GET /transactions/bulk...	50	1	0	7	1.10	0.00%	28.0/sec	867.51	5.46	31766.5
GET /transactions/bulk...	50	1	0	20	2.77	0.00%	27.9/sec	11.70	6.46	429.1
GET /transactions/bulk...	50	1	0	20	2.90	0.00%	28.0/sec	13.74	6.48	502.5
GET /demo/race-condi...	50	137	33	339	100.00	0.00%	24.7/sec	25.09	4.73	1039.0
DELETE /transactions/...	50	3	1	16	2.43	0.00%	27.9/sec	1.99	5.90	73.0
DELETE /transactions/...	50	3	1	9	1.64	0.00%	27.9/sec	1.99	5.89	73.0
DELETE /transactions/...	50	2	1	10	1.79	0.00%	28.0/sec	1.99	5.90	73.0
DELETE /categories/{id}	50	2	1	8	1.42	0.00%	28.0/sec	2.00	5.91	73.0
DELETE /budgets/{id}	50	3	1	10	1.78	0.00%	28.0/sec	1.99	5.76	73.0
DELETE /accounts/{id}...	50	2	1	13	1.95	0.00%	30.5/sec	2.17	6.37	73.0
DELETE /accounts/{id}...	50	3	1	30	5.80	0.00%	34.6/sec	2.47	7.23	73.0
DELETE /users/{id}	50	4	1	31	5.92	0.00%	41.5/sec	2.96	8.55	73.0
DEMO ERROR GET /us...	1	12	12	12	0.00	100.00%	83.3/sec	32.88	15.06	404.0
TOTAL	2101	11	0	339	32.73	0.05%	47.8/sec	229.10	12.54	4910.0

Рисунок 4.2 – Результаты нагрузочного тестирования

5 ЗАКЛЮЧЕНИЕ

В ходе работы исследованы проблемы многопоточности: race condition. Реализовано создание потоков через ExecutorService, продемонстрирована работа synchronized и Atomics для атомарности операций.